Aprendizagem Aplicada à Segurança

Class 6: Anomaly Detection II, autoencoders, time, space and the state of the art

Mário Antunes

October 23, 2026

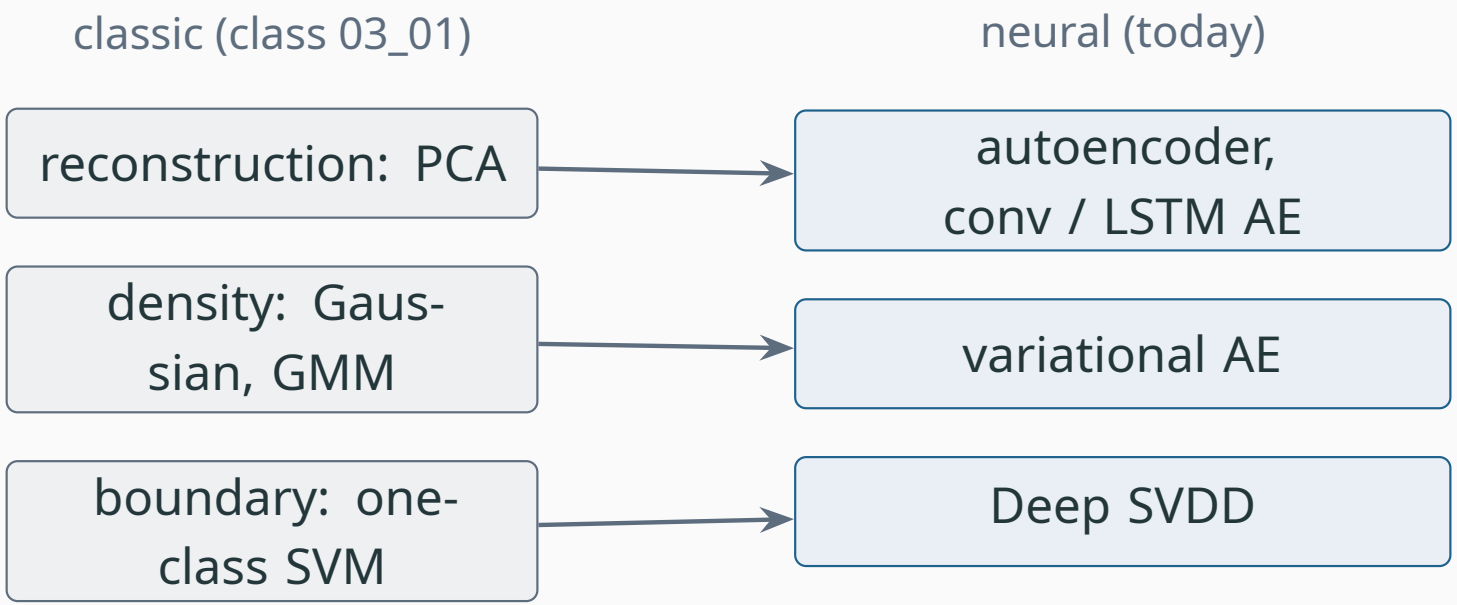
Universidade de Aveiro

Class 03_01 in one frame

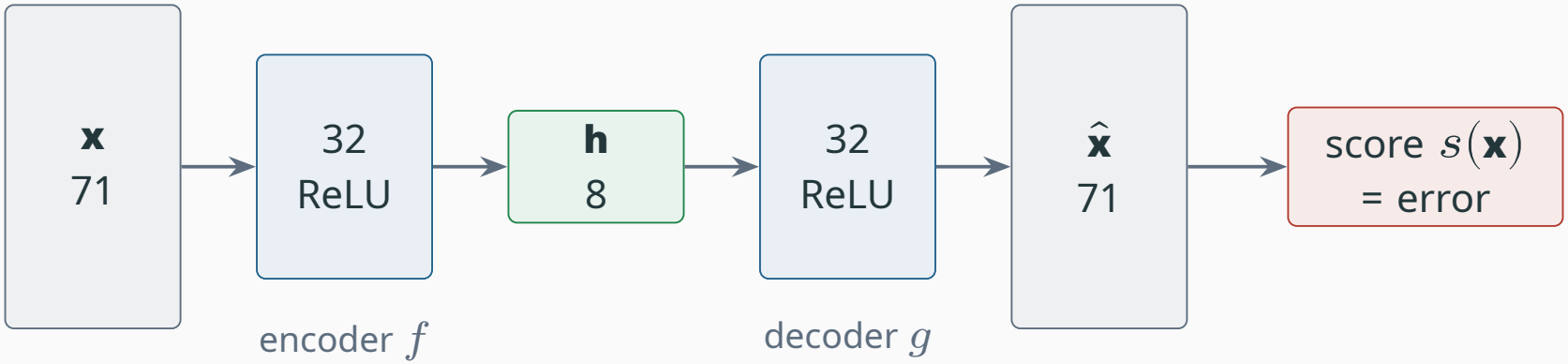
- Model **normal**, score the departure $s(\mathbf{x})$, set t at 1% false alarms on normal validation data.
- KDD99, 1.8% attacks: ten classic detectors reach AUPRC 0.33 to 0.68.
- The GMM is best; PCA has the best F1.
- r2l looks like normal traffic: every detector struggles; a classifier misses attack kinds it never saw.
- Today: can **learned representations** do better, and what about anomalies in **time** and across **sensors**?

Why neural detectors?

- Hand-made features work for tables like KDD99; they are poor for raw **sequences**, **images**, **many sensors**.
- A neural network learns the features from normal data; the price: tuning, training time, less transparency.
- Deep and shallow detectors share a few principles (Ruff et al. 2021):

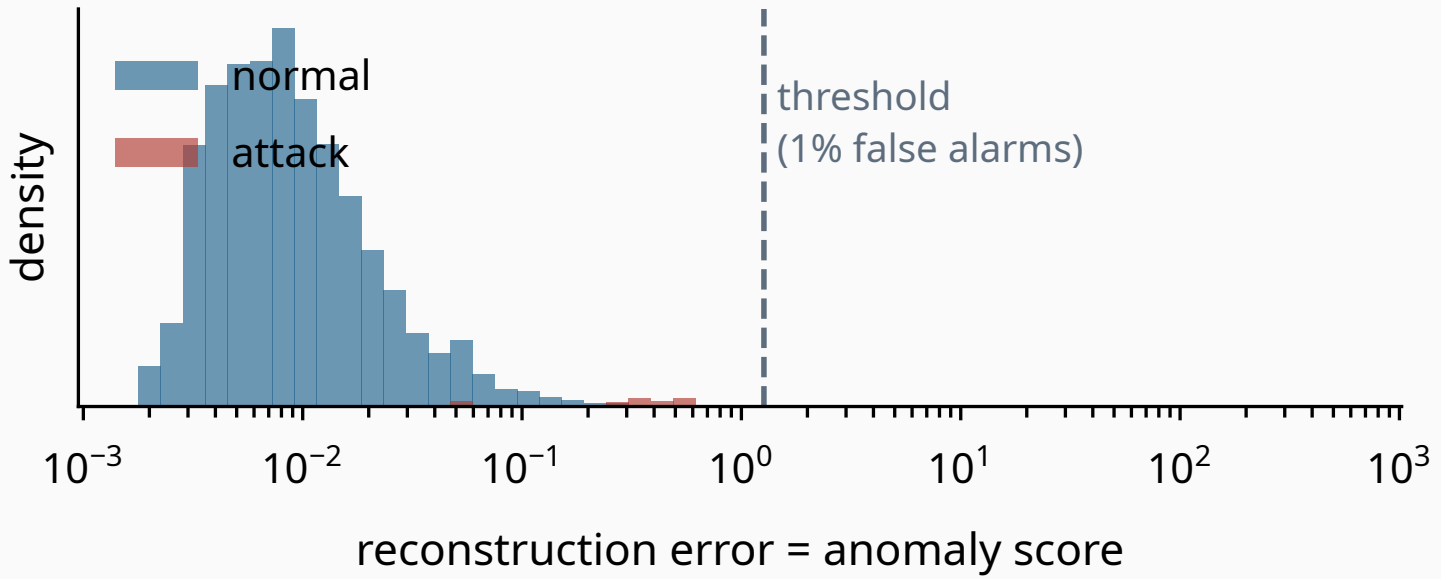


The autoencoder: squeeze, then rebuild



- Train $\hat{\mathbf{x}} = g(f(\mathbf{x}))$ on **normal** rows to minimise $\frac{1}{n}\sum_i \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2$.
- The 8-unit **code h** cannot copy 71 numbers: it keeps the structure of normal data only.

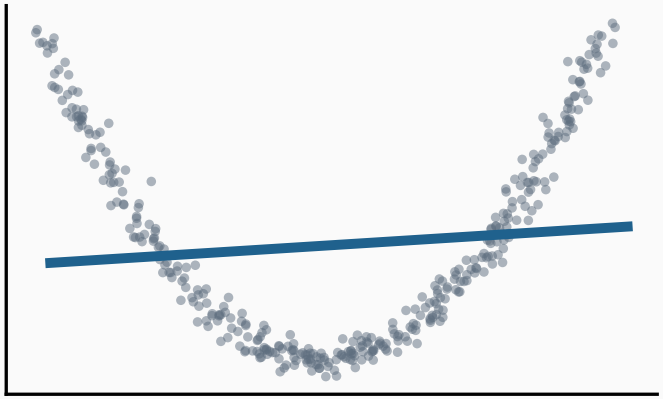
Normal rebuilds well, attacks do not



- Test set, 1.8% attacks: the score is the reconstruction error; attacks sit to the right of the threshold.
- The threshold is the 99% quantile of the validation scores of normal rows.

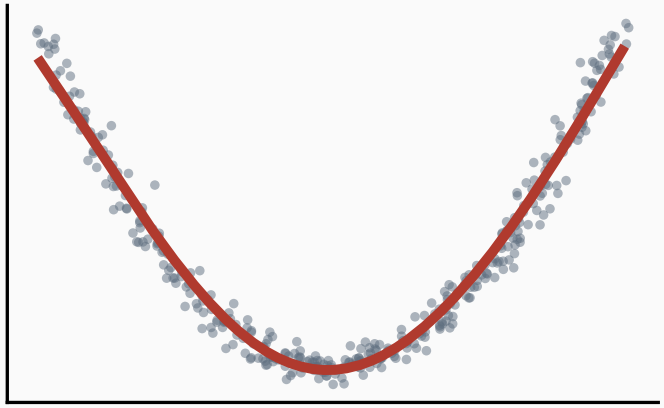
A linear autoencoder is PCA

linear code: PCA



mean squared error 0.215

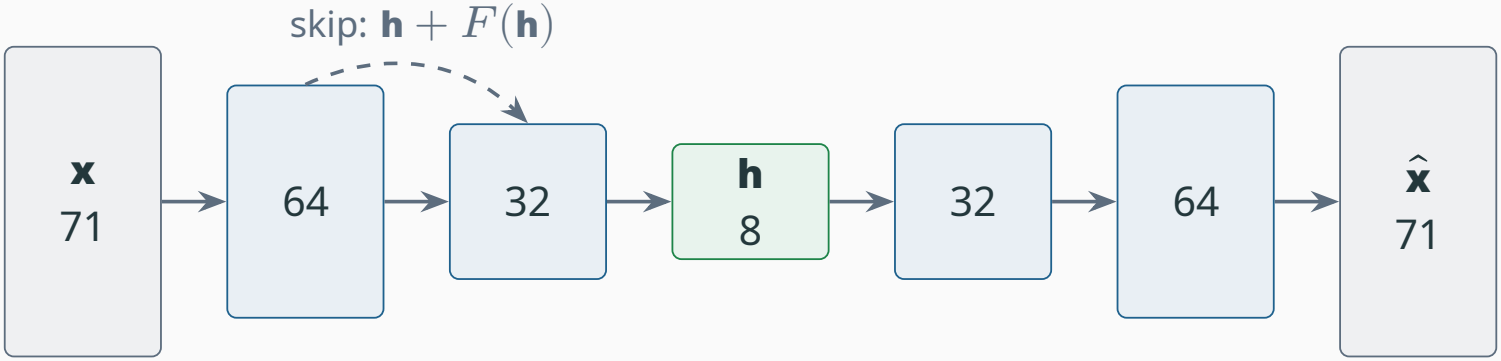
AE: code of 1 unit, tanh layers



mean squared error 0.003

- Without activations, $\hat{\mathbf{x}} = W_2 W_1 \mathbf{x}$ and the best rank-k map is the PCA projection $V_k V_k^\top$ (Baldi and Hornik 1989).
- The ReLU or tanh layers bend the subspace into a curved surface: the only gain over PCA.

Deeper: a funnel, SiLU and skip connections



- **Funnel**: each layer narrows the previous one; a second hidden layer is the step that pays (next results).
- **SiLU** $x\,\sigma(x)$ is a smooth ReLU; ASAP 2.0 uses two such layers per side (Silva et al. 2025).
- A **skip** adds the block input to its output; layer normalisation keeps deep stacks stable.

One function builds them all

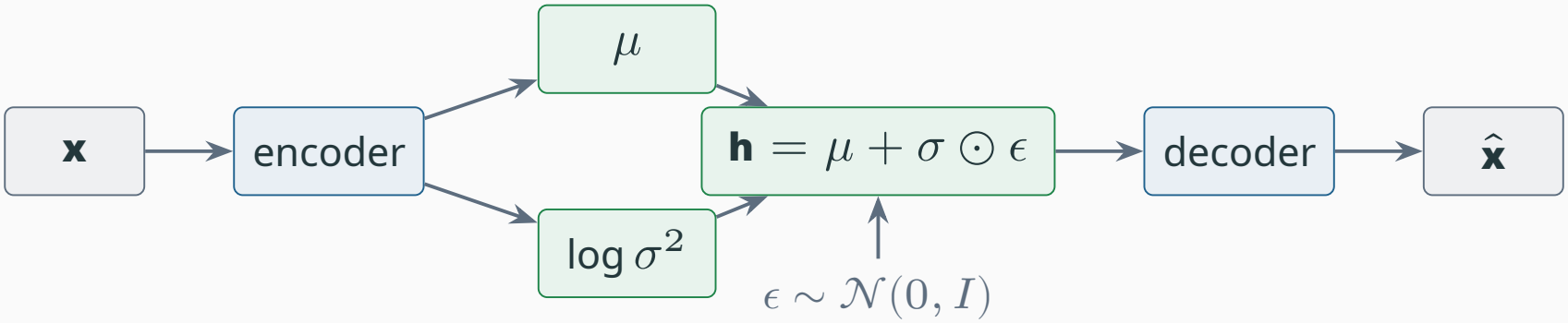
```
model = build_autoencoder(
    (71,), kind="dense",           # "conv" | "lstm" | "gru" for windo
    widths=(64, 32), code=8,       # funnel and bottleneck
    activation="silu", norm="layer", residual=True,
    l1=0.0, noise=0.0)             # sparse / denoising
fit_autoencoder(model, X_normal)  # early stopping on 10% of the rows
err = reconstruction_error(model, X)   # per row, step or feature
```

- One builder in `src/autoenc.py` serves tabular rows and windows of a series, with or without skips.
- Training uses normal rows only; the held-out 10% stops training before the AE over-fits.

Terminology: variations on the autoencoder

Model	Change	Why
residual AE	block $\mathbf{h} + F(\mathbf{h})$ (He et al. 2016)	stable deep stacks
sparse AE	$+\lambda\|\mathbf{h}\|_1$ on the code	few active units, less copying
denoising AE	input $\mathbf{x} + \epsilon$, target $\mathbf{x}$ (Vincent et al. 2008)	learns the shape, not the points
variational AE	code is a distribution (Kingma and Welling 2014)	a density, not only a surface
Deep SVDD	no decoder; pull $\phi(\mathbf{x})$ to a centre (Ruff et al. 2018)	one-class boundary in learned space

Variational AE: a code with uncertainty



- The encoder outputs a mean μ and a log-variance; the code is **sampled** around the mean.
- Score: the reconstruction error. In practice clip $\log \sigma^2$ and the gradient norm, or a seed can diverge (it did here).

Variational AE: definition of the loss

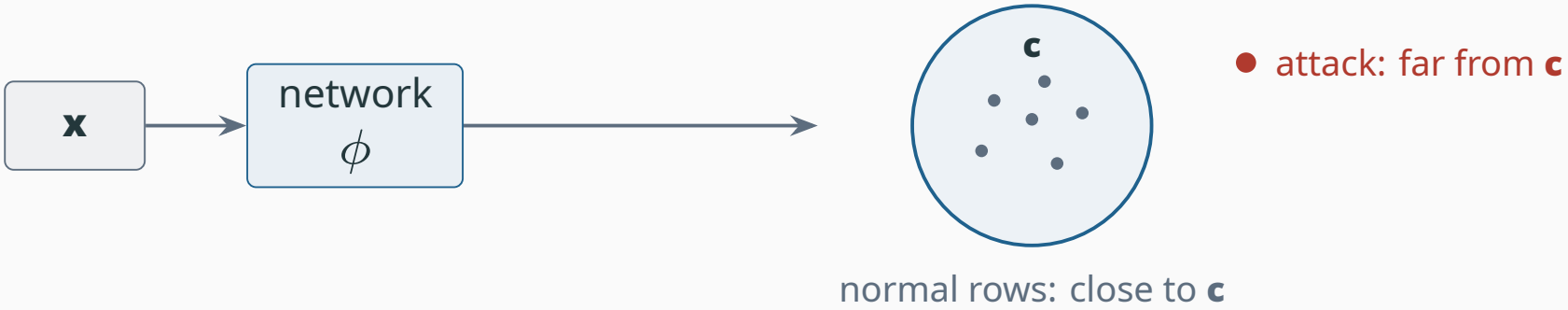
- Loss = reconstruction + β times the distance of the code distribution from $\mathcal{N}(0, I)$:

$$\mathcal{L} = \|\mathbf{x} - \hat{\mathbf{x}}\|^2 + \beta \,\mathrm{KL} \quad \text{(definition)}$$

$$\mathrm{KL} = -\frac{1}{2}\sum_j \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right) \quad \text{(definition)}$$

- The sum runs over the code units j. Why the sampling is differentiable: appendix.

Deep SVDD: a sphere in a learned space



- A network ϕ maps normal rows close to a fixed centre **c**; the score is the distance to it.
- Deep SAD adds a few labelled anomalies that are pushed *away* from **c** (Ruff et al. 2020).

Deep SVDD: definition and collapse

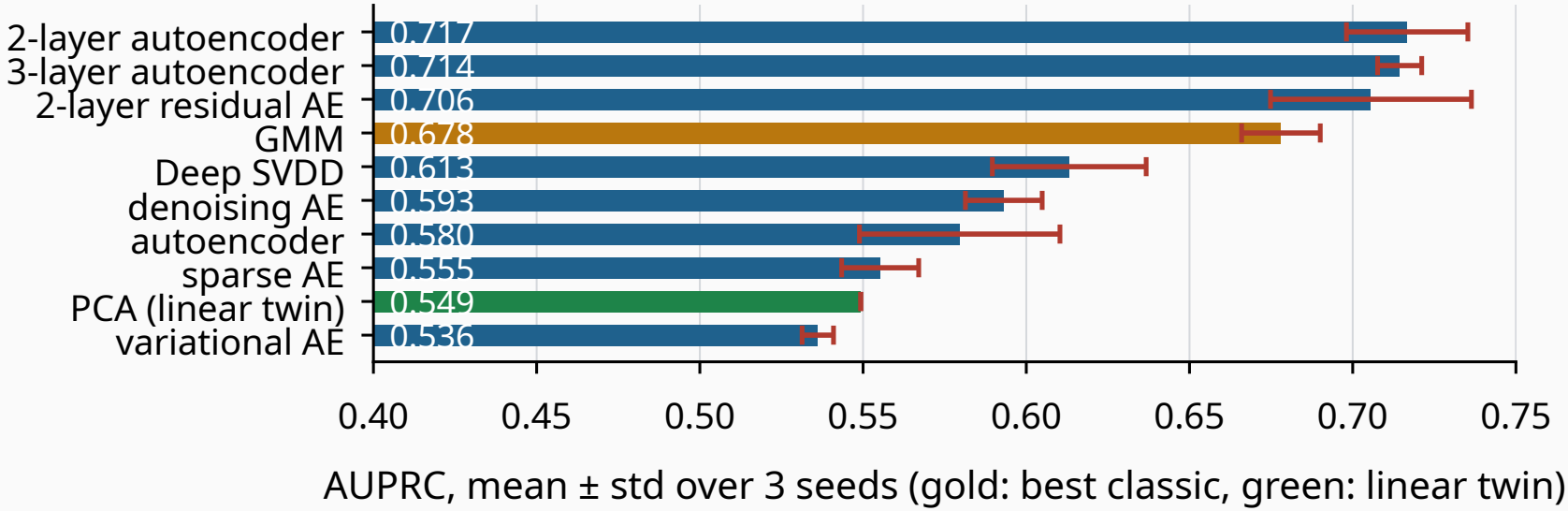
$$\min_{\phi} \frac{1}{n} \sum_{i} \|\phi(\mathbf{x}_i) - \mathbf{c}\|^2, \qquad s(\mathbf{x}) = \|\phi(\mathbf{x}) - \mathbf{c}\|^2 \quad \text{(definition)}$$

- $\mathbf{c}$ is the mean of the initial outputs, then fixed.
- **Collapse:** if ϕ may output a constant, $\phi(\mathbf{x}) = \mathbf{c}$ for every $\mathbf{x}$ is a perfect, useless solution.
- Remedies of the paper: no bias terms, no bounded activations (Ruff et al. 2018).

DEMO 04

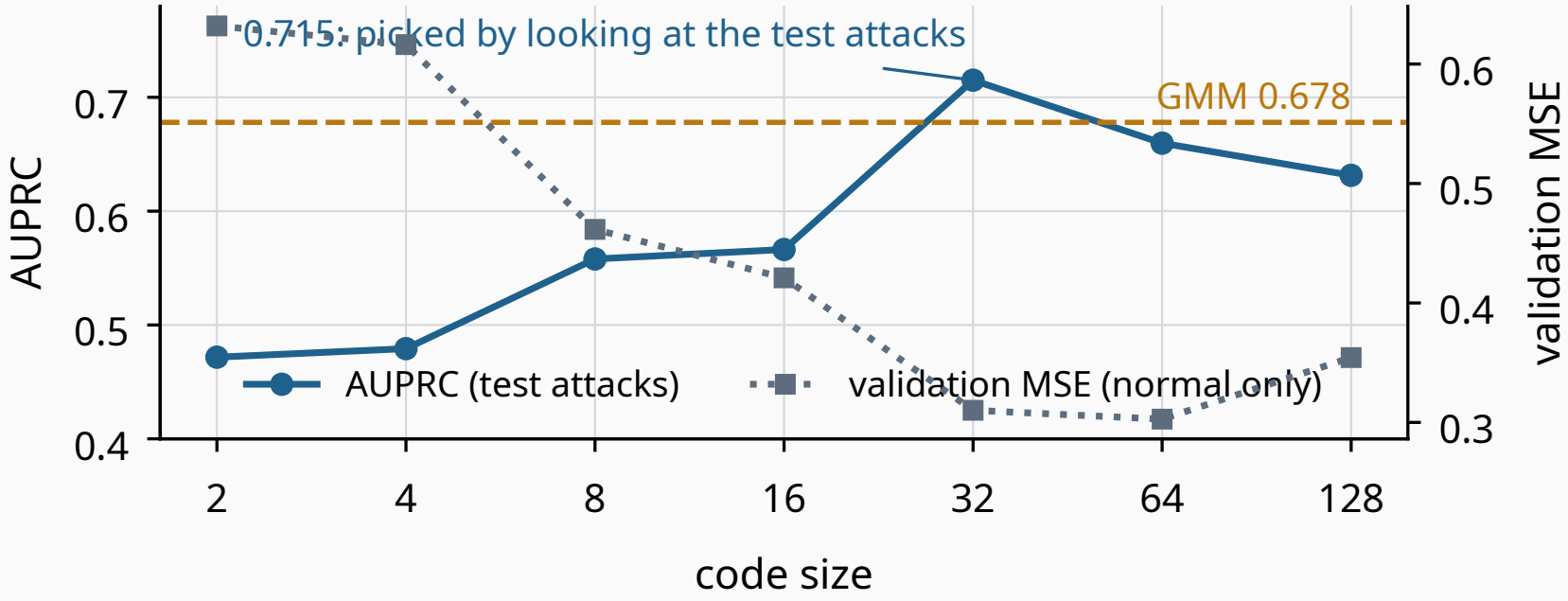
`notebooks/02-anomaly/notebook_04.ipynb`

Depth moves the needle



- 3 seeds: a second hidden layer lifts the AE from 0.58 to 0.72, above the GMM (0.68) and PCA (0.55).
- A third layer or skips add nothing (0.71) and cost 4x the time.
- Sparse, denoising and variational codes hurt (0.54 to 0.59).

How big should the code be?

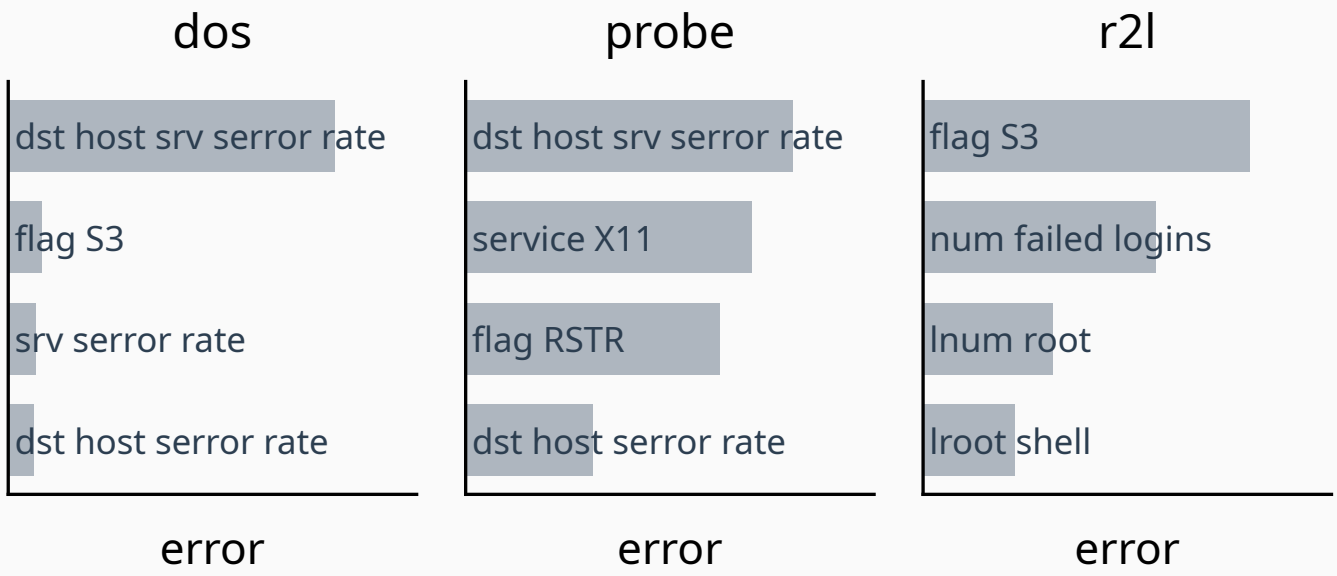


- Too small: normal traffic does not fit. Too large: the AE starts to copy attacks too.
- **Who chose 32?** We looked at the test attacks. The validation MSE of normal rows prefers 64.

Tuning without anomalies is guessing

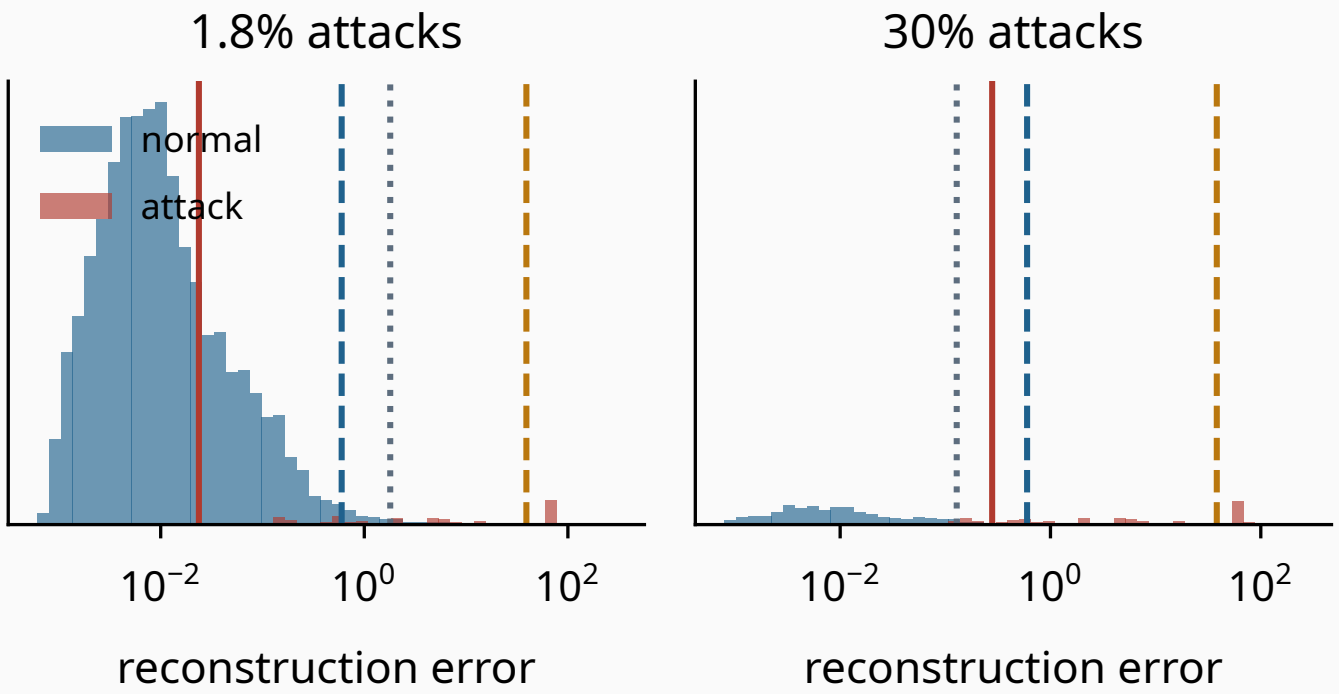
- A one-class detector has no labelled anomalies to tune code size, epochs, ν, k on.
- A gain from tuning on the test attacks is **not** a gain.
- Report the default, or tune on a separate labelled set.
- Options: a few labelled anomalies (semi-supervised), a simulated attack set, or robust defaults over seeds.

Explaining an alarm: error per feature



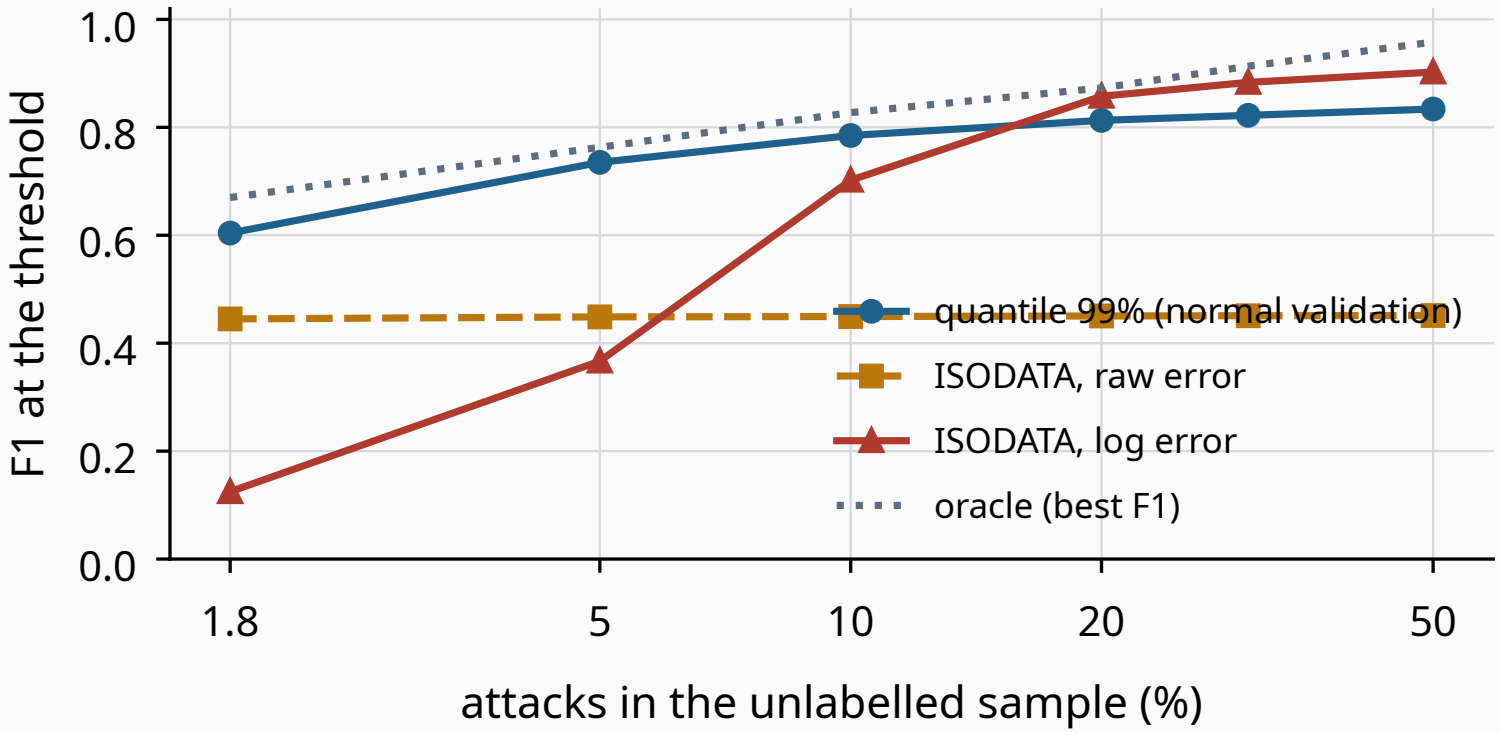
- The reconstruction error is a sum over features: the largest terms name the reason.
- A triage hint: SYN-error rates for floods, failed logins and shells for intrusions.

A threshold without labels: ISODATA



- **ISODATA** (Ridler and Calvard 1978): start at the mean error, then $T \leftarrow \frac{1}{2}(\bar{e}_{\leq T} + \bar{e}_{>T})$ until it stops.
- No label is used; ASAP 2.0 applies it to AE errors (Silva et al. 2025). It assumes two modes.

It needs a second mode

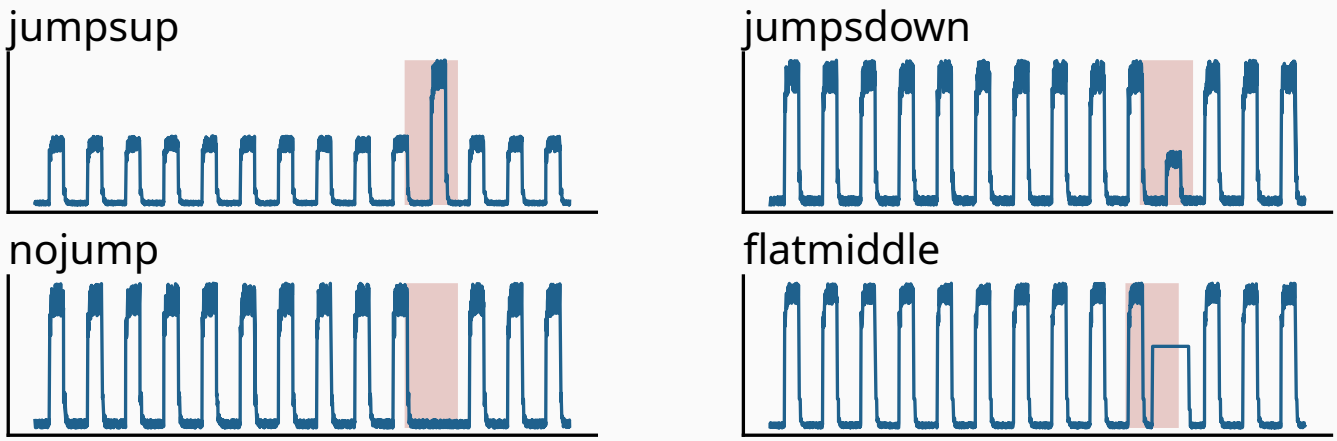


- At 1.8% attacks the log-error cut fails (F1 0.12; the quantile gets 0.60); from 10% it follows the oracle.
- On the raw error it never works (F1 0.45): the heavy tail drags the means. Take the log.

Which rule, when?

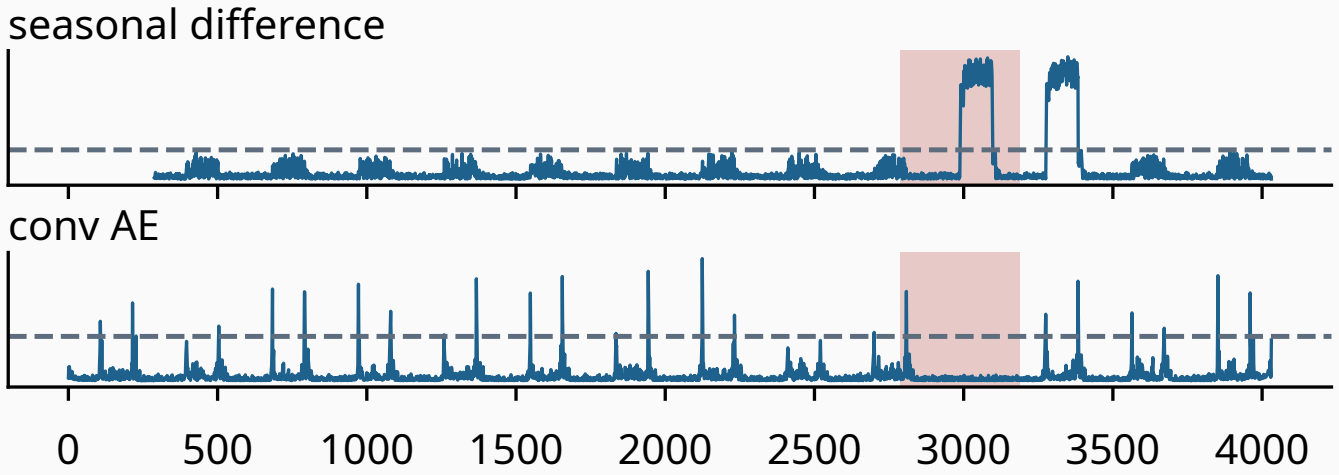
- **Quantile of clean validation errors**: stable at any base rate, but needs clean data and fixes the false-alarm rate.
- **ISODATA on log errors**: no clean data, but only when attacks are frequent (campaigns, monitoring).
- The paper balanced its test set by undersampling; a real network is at 1% or less.
- Either way, the data that sets the threshold is part of the attack surface (class 03_03).

A value can be normal and still wrong *now*



- A **contextual** anomaly (80 requests/s: normal at noon, not at 3 a.m.) needs a **window**.
- Only jumpsup leaves the normal range; jumpsdown, nojump (no daily rise) and flatmiddle stay inside it.
- Four NAB series (Lavin and Ahmad 2015), 14 days, red = anomaly: train on days 1-8, threshold = max score of day 9 + 10%.

Three one-liners, one autoencoder



time step (5 min); red: the anomaly window, dashed: threshold

- One-liners: $|x_t|$; **seasonal difference** $|x_t - x_{t-288}|$; moving std. **Conv AE**: windows of 288 steps.
- AE score of step t: the squared error **at** t, averaged over the windows holding it.
- Plot: the seasonal difference (top) echoes a day later, the conv AE does not.

DEMO 05

notebooks/02-anomaly/notebook_05.ipynb

Found, and at what price

	|x|	seasonal diff	moving std 1h	conv AE
jumpsup	found, 0	found, 114	found, 1	found, 0
jumpsdown	missed	found, 112	missed	found, 3
nojump	missed	found, 108	missed	found, 35
flatmiddle	missed	found, 355	missed	found, 1

- Each cell: found or missed, then false alarms outside the NAB window.
- The seasonal difference finds all four but echoes (~110 false alarms); the conv AE finds all four with 0 to 35.
- An LSTM AE over 8-hour windows **misses** nojump: the window must hold the context.

Which body? Not the fanciest

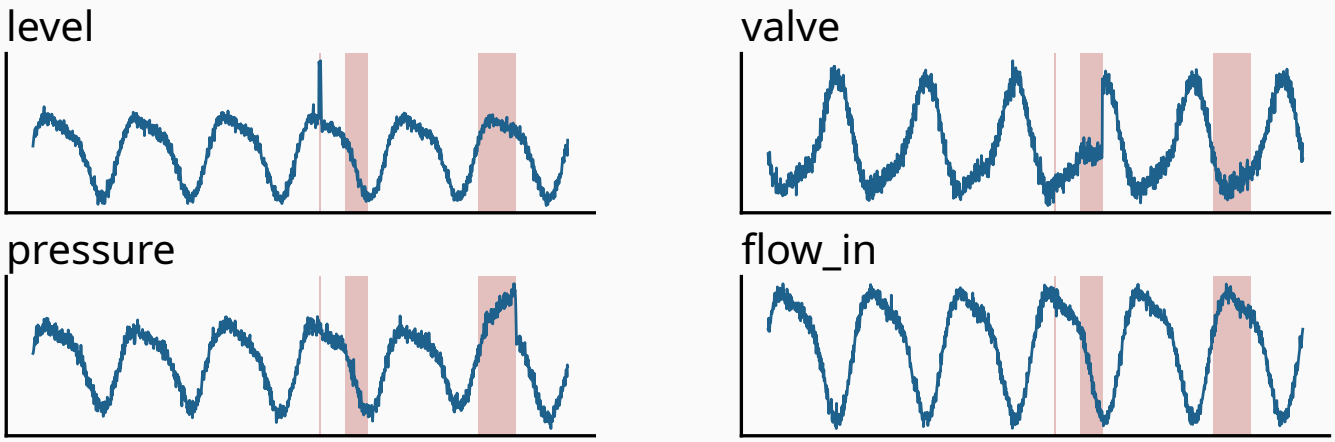
Body (window)	Found of 4	False alarms
conv, 2 stages, ReLU (1 day)	4	10
conv, 2 stages, SiLU (1 day)	2	5
residual conv, 3 stages (1 day)	1	0
GRU (8 h)	1	4

- Same NAB protocol; the plain ReLU conv AE wins: a smoother or deeper body rebuilds the anomaly too.
- ISODATA on the log score finds all four but raises 990 to 2,700 false alarms: a series is not bimodal.

The benchmarks are telling us something

- Several popular time-series benchmarks are solvable by one-liners, and their labels have errors (Wu and Keogh 2021).
- TSB-AD (40 data sets, 40 methods): simple and statistical methods are often as good as deep ones (Liu and Paparrizos 2024).
- Time-series **foundation models** (pre-trained on many series, used zero-shot): no better than moving-window one-liners for anomaly detection.
- Lesson: always run the one-liners first; a deep model must beat them on *your* data.

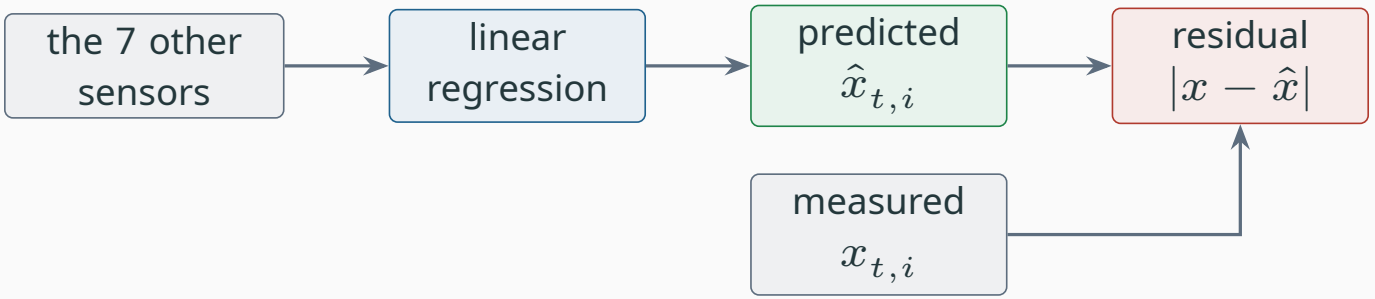
Example 2: a small water plant



- 8 sensors, two hidden processes: **demand** (daily cycle), **heat** (slow wander); correlated in **space** and **time**.
- Events in the second half (red): **spike** on level (+1.2, 30 min); **drift** of pressure (slow ramp, 10 h).
- **Freeze** of the valve: a stale value for 6 h, every value normal.

Graphs of sensors: predict each one from the others

- Graph-based detectors learn which sensors depend on which and alarm when one departs from what its neighbours predict (GDN (Deng and Hooi 2021)).



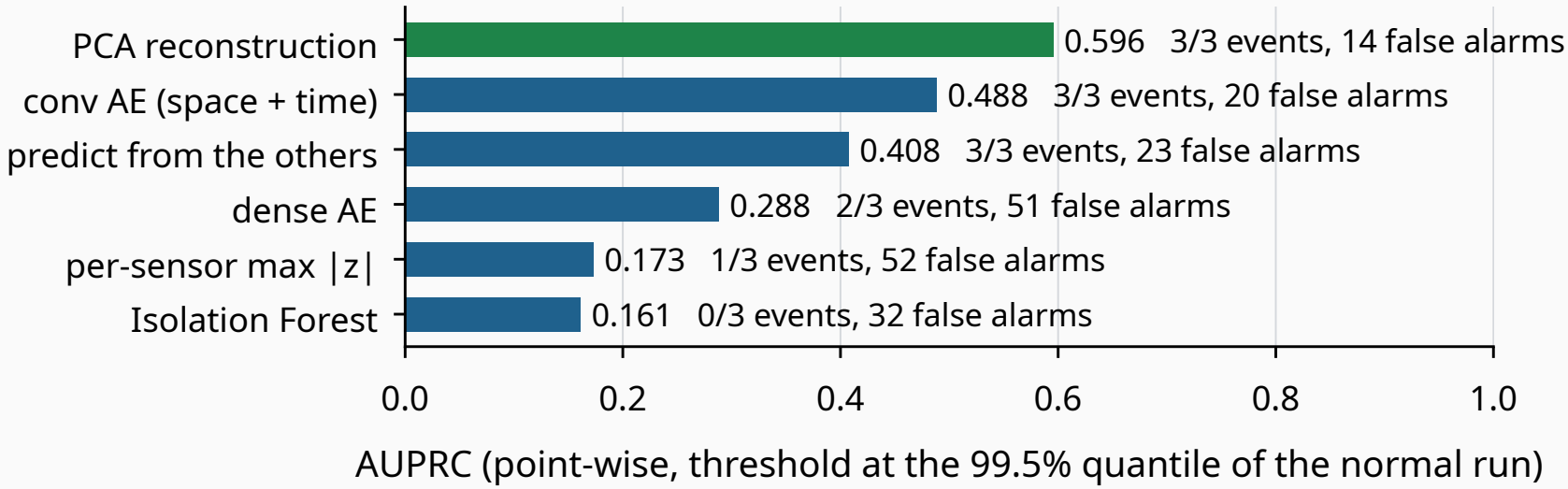
$$s_t = \max_i \left| x_{t,i} - \hat{x}_{t,i}(\mathbf{x}_{t,-i}) \right| \quad \text{(definition)}$$

- A frozen valve reports a normal value, but not the one its neighbours predict.

DEMO 06

`notebooks/02-anomaly/notebook_06.ipynb`

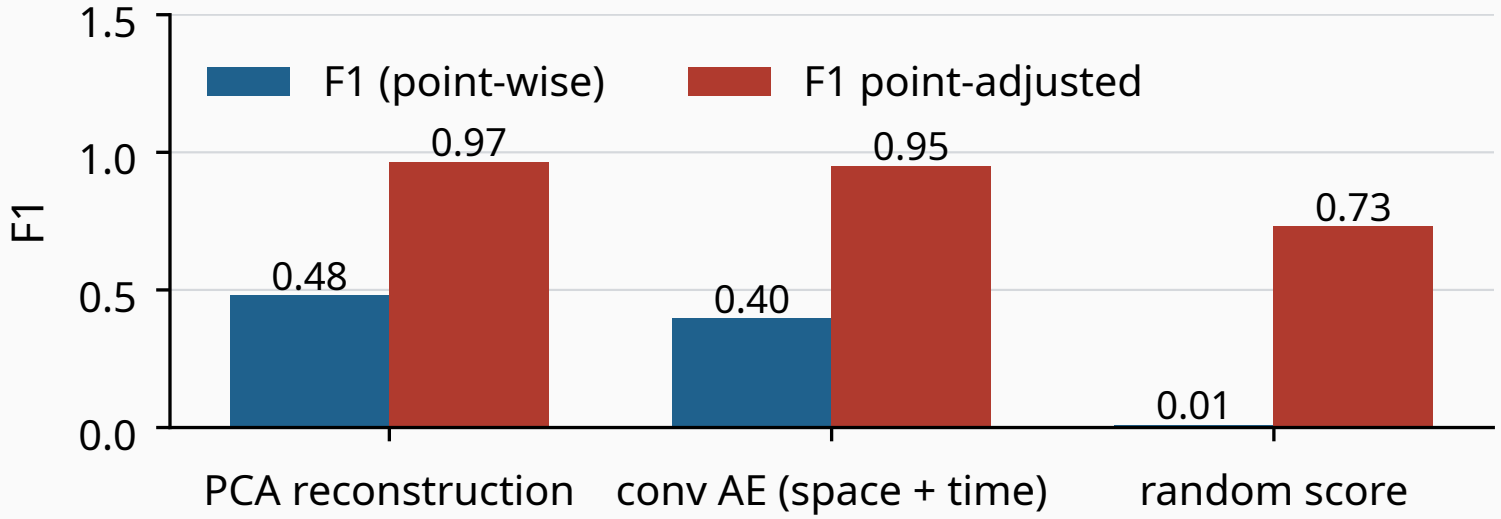
Space beats per-sensor checks



- The freeze and the drift break **relations**, not ranges: only models of the relations see them.
- With smooth, strongly correlated sensors the linear model wins; time pays off when the anomaly is in the *shape* (NAB).

The point-adjust trap

- **Point-adjusted** F1 (many papers): one flagged point counts the whole anomalous segment as found.



- A random score looks almost as good as the best detector (Kim et al. 2022).
- Report point-wise metrics, events found, delay, false alarms, baselines.

A map of the newer tools

Kitsune
online ensemble of small AEs on packet statistics
assumes live traffic is clean

GDN
graph of sensor dependencies
names the sensor

TranAD
transformer reconstruction
gains on benchmarks with known flaws

Foundation models
pre-trained fore-casters, zero-shot
about as good as one-liners

- Sources: Kitsune (Mirsky et al. 2018), TranAD (Tuli et al. 2022); GDN (Deng and Hooi 2021).
- Each brings a real idea; none removes the base rate, the threshold, or the need for baselines.

Autoencoder families in the literature

Family	Idea	Fixes
DAGMM (Zong et al. 2018)	code and error feed a GMM	a density threshold
MemAE (Gong et al. 2019)	code = mix of stored normal prototypes	AE rebuilds anomalies
USAD (Audibert et al. 2020)	two AEs, trained adversarially	weak reconstruction gap
Anom. Transformer (Xu et al. 2022)	attention discrepancy over windows	context beyond a window

- Recurring problem: an AE that **generalises** so well it rebuilds the anomaly.

What to take from the literature

- No detector wins everywhere, on tables (Han et al. 2022) or on series (Liu and Paparrizos 2024).
- Evaluation choices (duplicates, point adjust, tuning on the test set) move results more than architectures.
- A good deep detector is a good **representation** plus the same protocol as class 03_01.
- Next class: the same detectors as **targets**. What an attacker can change decides what they are worth.

Labs and practice guide

Notebook	Content
04	`build_autoencoder`: deep, residual, sparse, denoising, variational, Deep SVDD on KDD99; code size; ISODATA threshold
05	NAB: one-liners vs conv, residual conv and GRU AE; per-step scores; ISODATA on a series
06	Example 2: per-sensor vs space vs space + time; point-adjust; attribution

- All neural models: Keras 3 on the JAX backend, CPU is enough (notebooks 04-05 take a few minutes).
- **Guide** `practice/anomaly/guide_03_02`: a deep AE by hand in Keras, its PCA twin, an ISODATA threshold, a conv AE on NAB.

Take-aways

- An autoencoder is PCA with curves; a second hidden layer is what pays; Deep SVDD is a one-class boundary.
- A threshold without labels (ISODATA, on log errors) works only when attacks are frequent.
- On hand-made features, deep detectors match the classic ones; their tuning has no labels to rely on.
- Time: score windows, localise the error per step, and beat the seasonal one-liner first.
- Space: anomalies that break **relations** need a model of the relations (PCA, graphs, AEs over sensors).
- Distrust point-adjusted F1 and benchmark gains without baselines.

The reparameterisation trick

- Sampling $\mathbf{h} \sim \mathcal{N}(\mu, \operatorname{diag} \sigma^2)$ is not differentiable in μ, σ.
- Write $\mathbf{h} = \mu + \sigma \odot \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$: the randomness moves to an input, and $\partial \mathbf{h} / \partial \mu = I$, $\partial \mathbf{h} / \partial \sigma = \operatorname{diag} \epsilon$.
- KL of two Gaussians per code unit, closed form: $\mathrm{KL}(\mathcal{N}(\mu, \sigma^2) \| \mathcal{N}(0, 1)) = \frac{1}{2}(\mu^2 + \sigma^2 - \log \sigma^2 - 1)$.
- Summed over the units it gives the loss term of the VAE frame.

Why a linear AE recovers the PCA subspace

- Minimise $\sum_i \|\mathbf{x}_i - W_2 W_1 \mathbf{x}_i\|^2$ (centred data): $P = W_2 W_1$ has rank at most k.
- For a fixed rank-k projection P the error is $\mathrm{tr}(S) - \mathrm{tr}(PS)$, S the scatter matrix.
- $\mathrm{tr}(PS)$ is maximal when P projects onto the top k eigenvectors of S (Ky Fan): $P = V_k V_k^\top$.
- The AE finds the same subspace, not necessarily the same basis: W_1 can be any invertible mix of $V_k^\top$ (Baldi and Hornik 1989).

References (1/2)

Audibert, Julien, Pietro Michiardi, Frédéric Guyard, Sébastien Marti, and Maria A. Zuluaga. 2020. “USAD: UnSupervised Anomaly Detection on Multivariate Time Series.” ACM SIGKDD International Conference on Knowledge Discovery & Data Mining.

Baldi, Pierre, and Kurt Hornik. 1989. “Neural Networks and Principal Component Analysis: Learning from Examples Without Local Minima.” Neural Networks 2 (1).

Deng, Ailin, and Bryan Hooi. 2021. “Graph Neural Network-Based Anomaly Detection in Multivariate Time Series.” AAAI Conference on Artificial Intelligence.

Gong, Dong, Lingqiao Liu, Vuong Le, et al. 2019. “Memorizing Normality to Detect Anomaly: Memory-Augmented Deep Autoencoder for Unsupervised Anomaly Detection.” IEEE/CVF International Conference on Computer Vision (ICCV).

Han, Songqiao, Xiyang Hu, Hailiang Huang, Minqi Jiang, and Yue Zhao. 2022. “ADBench: Anomaly Detection Benchmark.” Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks.

He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. “Deep Residual Learning for Image Recognition.” IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

Kim, Siwon, Kukjin Choi, Hyun-Soo Choi, Byunghan Lee, and Sungroh Yoon. 2022. “Towards a Rigorous Evaluation of Time-Series Anomaly Detection.” AAAI Conference on Artificial Intelligence.

Kingma, Diederik P., and Max Welling. 2014. “Auto-Encoding Variational Bayes.” International Conference on Learning Representations (ICLR).

Lavin, Alexander, and Subutai Ahmad. 2015. “Evaluating Real-Time Anomaly Detection Algorithms – the Numenta Anomaly Benchmark.” IEEE International Conference on Machine Learning and Applications (ICMLA).

Liu, Qinghua, and John Paparrizos. 2024. “The Elephant in the Room: Towards a Reliable Time-Series Anomaly Detection Benchmark.” Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks.

Mirsky, Yisroel, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. 2018. “Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection.” Network and Distributed System Security Symposium (NDSS).

References (2/2)

Ridler, T. W., and S. Calvard. 1978. “Picture Thresholding Using an Iterative Selection Method.” IEEE Transactions on Systems, Man, and Cybernetics 8 (8): 630–32.

Ruff, Lukas, Jacob R. Kauffmann, Robert A. Vandermeulen, et al. 2021. “A Unifying Review of Deep and Shallow Anomaly Detection.” Proceedings of the IEEE 109 (5).

Ruff, Lukas, Robert A. Vandermeulen, Nico Görnitz, et al. 2020. “Deep Semi-Supervised Anomaly Detection.” International Conference on Learning Representations (ICLR).

Ruff, Lukas, Robert Vandermeulen, Nico Görnitz, et al. 2018. “Deep One-Class Classification.” International Conference on Machine Learning (ICML).

Silva, Catarina, João Felisberto, João Paulo Barraca, and Paulo Salvador. 2025. “ASAP 2.0: Autonomous & Proactive Detection of Malicious Applications for Privacy Quantification in 6G Network Services.” Computer Communications 237: 108145.

Tuli, Shreshth, Giuliano Casale, and Nicholas R. Jennings. 2022. “TranAD: Deep Transformer Networks for Anomaly Detection in Multivariate Time Series Data.” Proceedings of the VLDB Endowment 15 (6).

Vincent, Pascal, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. 2008. “Extracting and Composing Robust Features with Denoising Autoencoders.” International Conference on Machine Learning (ICML).

Wu, Renjie, and Eamonn J. Keogh. 2021. “Current Time Series Anomaly Detection Benchmarks Are Flawed and Are Creating the Illusion of Progress.” IEEE Transactions on Knowledge and Data Engineering.

Xu, Jiehui, Haixu Wu, Jianmin Wang, and Mingsheng Long. 2022. “Anomaly Transformer: Time Series Anomaly Detection with Association Discrepancy.” International Conference on Learning Representations (ICLR).

Zong, Bo, Qi Song, Martin Renqiang Min, et al. 2018. “Deep Autoencoding Gaussian Mixture Model for Unsupervised Anomaly Detection.” International Conference on Learning Representations (ICLR).